

First instruction trace with Ripes

Lab 0 · Computer Organization · Week 1

Name: Wazed Md Abdul (瓦责德) Student ID: 301202424031149

Class / group: 2024/ Software engineering Date: 24/09/2026

Purpose

Connect the first week's diagrams to a running processor model. Predict how three instructions change the program counter and registers, check the prediction in Ripes, and relate a cycle count to modeled CPU time.

Learning outcomes

By the end of the lab, you should be able to identify the PC, register file, ALU and instruction memory; step through RV32I code; record state changes; explain what the example does; and use $\text{CPU time} = \text{IC} \times \text{CPI} \times \text{cycle time}$.

Requirements and scope

Use a laptop with the instructor-approved Ripes desktop build or the experimental browser build linked from the official project. No GPU, CUDA installation or C compiler is required. RISC-V assembly syntax is introduced only as needed. [R1–R3]

Timing assumption

Two 45-minute academic periods = 90 minutes of activity, excluding the break. For two 50-minute periods, use the additional 10 minutes for setup support and the optional checks.

Time	Activity	Worksheet
0–10 min	Configure the single-cycle model	Page 2
10–30 min	Predict and trace Program A	Page 3
30–45 min	Follow the data through the CPU	Page 4
45–65 min	Run three controlled changes	Page 5
65–80 min	Connect cycles to time	Page 6
80–90 min	Reflect and prepare submission	Page 7

Student pages: 1–8. Pages 9–10 contain a detachable instructor answer key.

Configure Ripes before running code

A1. Open an approved build

Desktop: obtain a build for your operating system from the official Ripes Releases page, or use the version supplied by your instructor. Browser: the official project currently links to <https://ripes.dk/> and labels the browser deployment experimental. Internet access and JavaScript are needed for that route. Save your source locally. [R1, R3]

A2. Select the model

Use Select Processor (the processor icon). Choose the RV32 RISC-V Single Cycle Processor, not a five-stage or dual-issue model. Select Standard layout for the first trace. If ISA extension checkboxes are shown, disable C (compressed instructions). Use the base integer instruction set for this lab. [R2]

Initialize x5, x6 and x7 to zero using the processor's register-initialization settings, then Reset. Confirm x0 reads as zero. Register aliases may be displayed: x5 = t0, x6 = t1, x7 = t2. Record the initial PC as P rather than assuming a particular starting address.

Setup item	Record / check
Ripes version or browser build	Ripes-v2.2.6-win-x86_64
Operating system / browser	Windows 10 pro
Selected model	RV32 RISC-V Single Cycle Processor
C extension disabled / not enabled	Yes
After Reset: x5, x6, x7	0 0 0
Initial PC = P	0

A3. Learn the controls

Control / view	Use in this lab
Editor	Enter assembly. Correct code is assembled into the simulator.
Processor / Registers	Observe PC, component connections and register values.
Clock / single-step	Advance the model by one clock cycle.
Reset	Return to the program entry point and reset state.
Memory	Inspect addresses and memory contents.
Run	Do not use for the required trace: the stop label is a self-loop.

Checkpoint before continuing

Ask the instructor to verify the single-cycle model. A pipeline model will not produce the same one-clock-per-instruction trace. View labels can vary across releases; use the official introduction for your build. [R2]

Trace Program A

Enter this code exactly. Save it as lab0_baseline.s. The final jump keeps the PC in a safe self-loop; it is not part of the three-instruction measurement.

```
.text
main:
    addi x5, x0, 10
    addi x6, x0, 20
    add  x7, x5, x6
stop:
    jal  x0, stop
```

Syntax: addi destination, source, constant; add destination, source1, source2. In RV32I, x0 always reads zero. The other names identify registers. [R4]

B1. Predict before executing

After the first three instructions, predict: x5 = 10 x6 = 20 x7 = 30

Which register do you expect to change at each step? Explain in one sentence.

Ans: Clock 1 changes **x5**, Clock 2 changes **x6**, and Clock 3 changes **x7**, because each instruction writes its result to the register specified as its destination.

B2. Record the actual state

Reset. Confirm x5–x7 are zero. In the Processor view use Clock three times, one click at a time. Record values after each clock. Use decimal values for registers and an address format for the PC. Do not count the self-loop.

After	Instruction just executed	PC	x5	x6	x7
Reset	None	P = 0	0	0	0
Clock 1	addi x5, x0, 10	4	10	0	0
Clock 2	addi x6, x0, 20	8	10	20	0
Clock 3	add x7, x5, x6	12	10	20	30

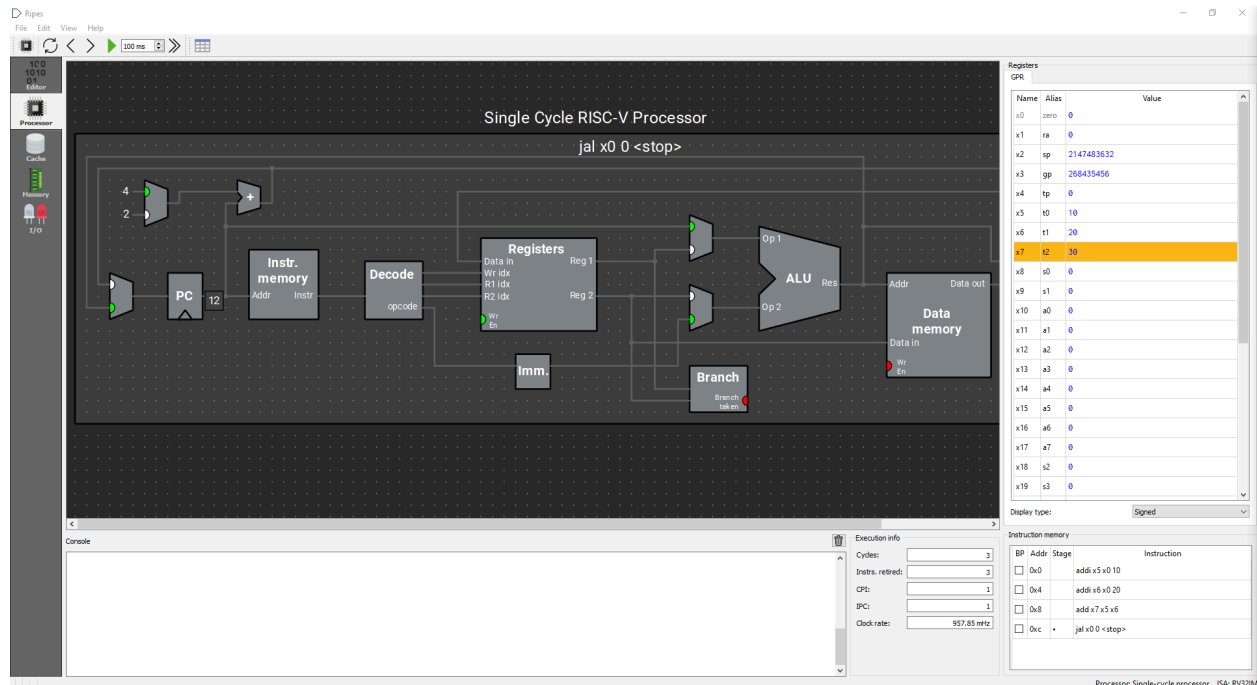
B3. Interpret the trace

How much did the PC increase after each of these instructions? Why?

Ans: The PC increased by 4 bytes after each instruction because each RV32I instruction used here is 32 bits, or 4 bytes, long.

Stop here

Save a screenshot of the state immediately after Clock 3. Do not press Run: the last instruction intentionally jumps to itself. If you step into the self-loop, Reset and repeat the three measured clocks.



Screenshot 1: Baseline after Clock 3.

PART C / DATAPATH OBSERVATION

Follow the data for add x7, x5, x6

Reset Program A and apply two clocks. The next arithmetic instruction is add x7, x5, x6. First inspect its input values, then apply the third clock and inspect the written result. If available, switch to Extended layout only for observation; keep the same single-cycle model. [R2]

C1. Identify the component roles

Component	What to observe	Your observation
PC	Address associated with the third instruction	8
Instruction memory	The instruction at that address	add x7, x5, x6

Register file	The two source-register values	x5 = 10, x6 = 20
ALU	Operation and computed result	ADD, result = 30
Destination register	Register that changes after Clock 3	X7
Data memory	Whether this instruction loads or stores data	No load or store

C2. Complete the data path

$x5 = 10$
 $x6 = 20$

 ALU operation: add $\rightarrow$ $x7 = 30$

The PC identifies an instruction address; it does not supply one of these two arithmetic operands. The register file supplies the operands for this add. Instruction fetch and data-memory access are different activities. [R4]

C3. Explain the result

1. Does add x7, x5, x6 change x5 or x6? Explain.

Ans: No. x5 and x6 do not change because they are source registers. Their values are read by the ALU, while x7 is the destination register that receives the result.

2. Does it need to load either operand from data memory? What was still fetched from instruction memory?

Ans: No. The operands are read from the register file, so data memory is not used. The instruction add x7, x5, x6 was still fetched from instruction memory.

PART D / CONTROLLED EXPERIMENTS

Change one thing at a time

Start each experiment from the original Program A, not from the previous variant. Save each variant, predict first, then Reset and use Clock. Leave the stop self-loop at the end.

D1. Change the input constant

Replace only the second arithmetic line with the following. Execute three measured instructions.

```
addi x6, x0, 7
```

D2. Change the operation

Restore the original constants 10 and 20. Replace only the third arithmetic line. Execute three measured instructions.

```
sub x7, x6, x5
```

D3. Try writing to the zero register

Restore all three original arithmetic lines. Insert this fourth line immediately before stop:. Execute four measured instructions, not three.

```
addi x0, x0, 99
```

Experiment	Prediction	Observed values	Match?
D1: after 3 clocks	x7 = 17	x6 = 7 x7 = 17	Yes
D2: after 3 clocks	x7 = 10	x7 = 10	Yes
D3: after 4 clocks	x0 = 0	x0 = 0 x7 = 30	Yes

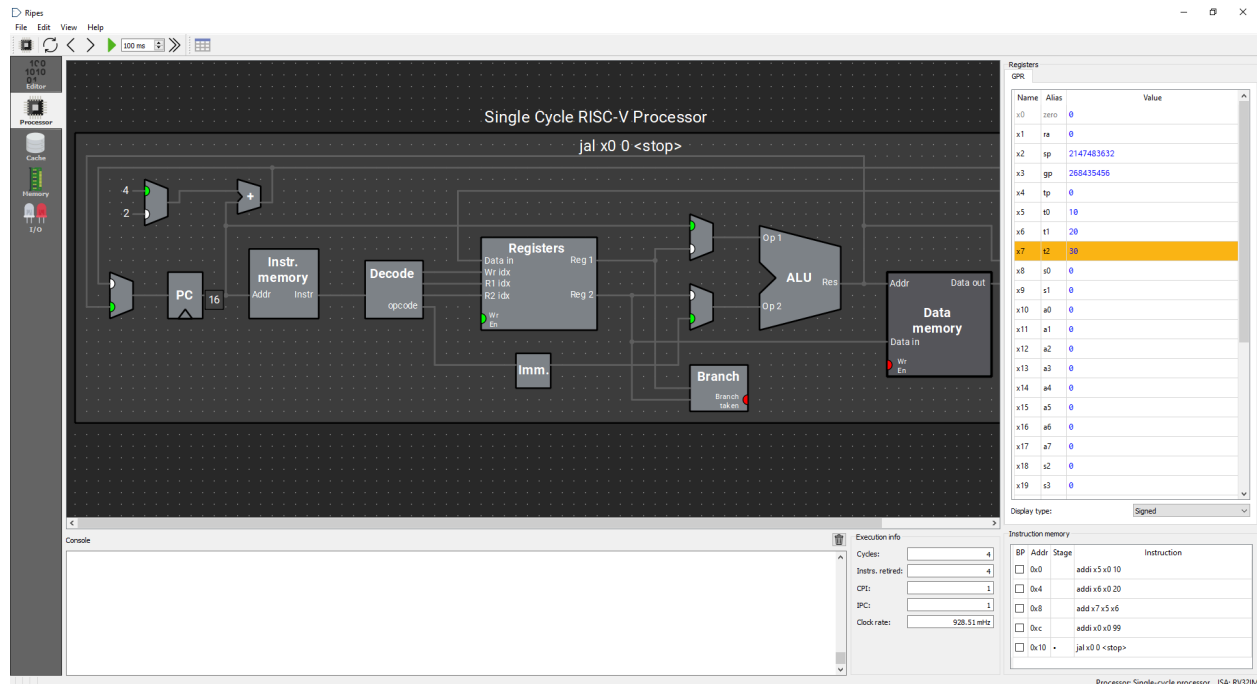
Explain one observation

Choose one experiment and explain the result using “source,” “destination” and either “constant” or “zero register.”

Ans: In D3, x0 is used as both a source and the destination. The constant 99 is added to the source value 0, but because the destination is the zero register, the result is discarded and x0 remains 0.

Reproducibility rule

Reset before every run. Record exactly which instructions were included. A self-loop instruction changes the cycle count even though the visible arithmetic result does not change.



Screenshot 2: D3 controlled experiment after Clock 4. The instruction attempts to write 99 to x0, but x0 remains 0.

PART E / PERFORMANCE

From simulated cycles to modeled time

Important distinction

Ripes is running on your real laptop or browser. Its animation interval and the time you spend clicking are not the simulated processor's hardware clock period. Use the explicitly assumed periods below; do not use a stopwatch on the interface. [R2]

For a defined instruction interval: $\text{cycles} = \text{IC} \times \text{CPI}$; modeled CPU time = $\text{cycles} \times T = (\text{IC} \times \text{CPI}) / f$. The examples below use fixed hypothetical clock rates. [M1, I1]

E1. Your three-instruction trace

Use only the first three arithmetic instructions in baseline Program A. In the selected single-cycle model, each of these instructions completes in one cycle. Assume $T = 2 \text{ ns}$ per simulated cycle.

Quantity	Your answer
Measured instruction count, IC	3 instructions
Modeled cycle count	3 cycles
Average CPI	1 cycles / instruction

Modeled CPU time	6 ns
Clock rate equivalent to $T = 2$ ns	0.5 GHz

E2. A larger hypothetical workload

A program executes 200 million instructions at average CPI = 1.5 on a processor with clock rate 2 GHz. Calculate CPU execution time in seconds. Show your units.

Ans: CPU execution time = 0.15 s

E3. Which processor is faster?

Two hypothetical processors execute the same workload with the same dynamic IC of 100 million instructions.

Processor	Clock rate	CPI	CPU time
A	2 GHz	1.0	0.05 s
B	3 GHz	2.0	0.0667 s

Faster processor: A. Speedup over the slower processor: 1.33 ×.

Can a higher clock rate alone determine the winner? Explain briefly.

Ans: No. A higher clock rate alone cannot determine the faster processor because CPU time also depends on instruction count and CPI.

PART F / REFLECT AND SUBMIT

Show evidence of understanding

F1. Evidence to submit

Submit your completed worksheet and four source files: lab0_baseline.s, lab0_D1.s, lab0_D2.s and lab0_D3.s. Include two readable screenshots as an appendix to your submission; do not shrink the register values until they are unreadable.

Screenshot	What it must show
1. Baseline after Clock 3	Selected single-cycle model, PC and x5/x6/x7.
2. One controlled change	Variant code, register result, and a caption naming the experiment.

F2. Short reflection

- Explain the difference between the PC and the value held in x7.

Ans: PC holds the address of the instruction being executed or fetched, while x7 is a general-purpose register that stores data or an arithmetic result.

- Which lab observation illustrates an ISA rule, rather than just a display choice?

Ans: The D3 experiment illustrates an ISA rule: x0 is the RISC-V zero register and always reads as zero, even when an instruction attempts to write another value to it.

- Why is a one-second animation interval not a one-second simulated hardware cycle?

Ans: A one-second animation interval is only the simulator's display speed. It does not represent the processor's hardware clock period, which is determined by the modeled cycle time.

F3. Suggested assessment rubric

Criterion	Points
Correct model selection and reproducible setup	2
Complete baseline trace with explanation	3
Predictions and observations for D1–D3	2
Performance calculations with units	2
Readable evidence and an independent explanation	1
Total	10

If Lab 0 is diagnostic in your course, use this rubric for feedback rather than a grade. The instructor sets the submission platform and deadline. Work may be discussed in pairs, but each student should produce their own trace and explanation.

SUPPORT / STUDENT REFERENCE

Troubleshooting and sources

Symptom	Check first
The editor reports an error	Use plain-text assembly; remove copied line numbers, smart punctuation and HTML tags such as <code>
</code> .
Registers change later than expected	Recheck that you selected the single-cycle model, not a pipeline model. Reset and repeat.
Program keeps running	The stop label is an intentional self-loop. Stop execution, Reset, and use Clock for the measured instructions.
My PC does not start at zero	Record its starting value as P. Compare increments relative to P.
PC increments differ from +4	Check C is disabled and inspect the assembled instruction list. Required examples use 32-bit RV32I instructions.
I see t0, t1 and t2	These are aliases for x5, x6 and x7. Numeric and alias names refer to the same registers.
The browser build does not load	Use the official desktop release or an instructor-managed lab computer. Do not disable system security controls.

My result changes after editing

Reset after each edit and verify the original input constants for that experiment.

Official resources and lecture references

[R1] Ripes project and desktop installation: <https://github.com/mortbopet/Ripes>

[R2] Ripes introduction, views and controls:
<https://github.com/mortbopet/Ripes/blob/master/docs/introduction.md>

[R3] Browser build (experimental): <https://ripes.dk/>

[R4] RISC-V RV32I specification: <https://docs.riscv.org/reference/isa/unpriv/rv32.html>

[M1] MIT 6.004, L21 Processor Performance:
<https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/pages/c21/c21s1/>

[I1] Imperial College, CPU law (printed slides 37–40; PDF page 10):
<https://www.doc.ic.ac.uk/~phjk/AdvancedCompArchitecture/2003-04/Lectures/Ch01/ACA-Ch01-CurrentVersion.pdf>

Sources checked 6 September 2026. Activities, code and worksheet wording are newly prepared. This introductory lab does not reproduce the later Vassar single-cycle control-signal lab. UI labels can change between Ripes builds.